

RECETARIO

Curso de Programación Python para IA

ÍNDICE

1. Introducción.....	3
2. Descripción del proyecto.....	3
3. Tecnologías utilizadas.....	3
4. Estructura del código	4
4.1 Archivo main.py	4
4.2 Archivo recetas.py.....	5
4.3 Archivo index.html	6
4.4 Archivo index.js.....	6
4.5 Archivo crear.html	7
4.6 Archivo crear.js.....	7
4.7 Archivo modificar.html	8
4.8 Archivo modificar.js.....	8
5. Funcionamiento del sistema	9
6. Capturas de pantalla	9
7. Consideraciones Adicionales	12
8. Bibliografía	13
9. Conclusión	14

1. Introducción

Este proyecto tiene como objetivo desarrollar una aplicación web que gestione y sugiera recetas de cocina a partir de un archivo CSV. La aplicación permite al usuario buscar recetas basadas en ingredientes específicos, así como crear, modificar y eliminar recetas a través de formularios. A partir de los ingredientes ingresados por el usuario, la aplicación busca en el CSV las recetas que contienen dichos ingredientes y las muestra al usuario.

2. Descripción del proyecto

El proyecto se compone de una aplicación web que interactúa con un archivo CSV que contiene información sobre diversas recetas. Los usuarios pueden buscar recetas ingresando uno o más ingredientes, y la aplicación utilizará un modelo de aprendizaje automático para sugerir las recetas más relevantes. Además, los usuarios tienen la capacidad de crear, actualizar y eliminar recetas directamente desde la aplicación, con los cambios reflejados en el archivo CSV. La aplicación también ofrece una lista de todos los ingredientes disponibles para facilitar la búsqueda y selección.

3. Tecnologías utilizadas

- **Python y Flask:** Implementación del backend y servidor web, gestionando las rutas y la lógica de la aplicación.
- **Pandas y Scikit-learn:** Manejo de datos en formato CSV y entrenamiento del modelo de recomendaciones basado en ingredientes.
- **JavaScript y HTML:** Creación de la interfaz de usuario y conexión con el backend mediante llamadas a la API.
- **CSV:** Almacenamiento persistente de las recetas, permitiendo su manipulación, consulta y actualización.

4. Estructura del código

4.1 Archivo main.py

- **Descripción:** Este archivo contiene el código del servidor Flask que procesa las peticiones del cliente, gestiona las recetas a partir de un archivo CSV y devuelve los datos en formato JSON.
- **Funcionamiento:**
 - **Configuración de la aplicación:** Inicializa la aplicación Flask y permite el acceso desde cualquier origen mediante CORS para facilitar las interacciones con el cliente.
 - **Rutas definidas para recetas:**
 - **/recetas (GET):** Devuelve todas las recetas almacenadas en el CSV, proporcionando un JSON con el número total de recetas y sus detalles.
 - **/recetas/<int:id> (GET):** Recupera una receta específica por su ID, devolviendo un JSON con la información de la receta o un error si no se encuentra.
 - **/recetas (POST):** Permite crear una nueva receta, tomando los datos del cuerpo de la petición, añadiéndola al CSV, y devolviendo la receta creada en formato JSON.
 - **/recetas/<int:id> (PUT):** Actualiza una receta existente según su ID. Acepta un JSON con los nuevos datos, actualiza la receta en el CSV y devuelve la receta actualizada.
 - **/recetas/<int:id> (DELETE):** Elimina una receta del CSV según su ID. Devuelve un mensaje de confirmación o un error si la receta no se encuentra.
 - **Búsqueda de recetas:**
 - **/sugerir_recetas (GET):** Recibe una lista de ingredientes como parámetros de consulta y utiliza el modelo de recomendaciones para sugerir recetas que coincidan.

Devuelve un JSON con el número de recetas sugeridas y sus detalles.

- **Obtención de ingredientes:**
 - **/ingredientes (GET):** Devuelve una lista de todos los ingredientes únicos disponibles en las recetas almacenadas en el CSV.

4.2 Archivo recetas.py

- **Descripción:** Controla la interacción del usuario con la interfaz de la página web.
- **Funcionamiento:**
 - **Carga y Guardado de Recetas:**
 - **cargar_recetas():** Carga las recetas desde un archivo CSV y las convierte a una lista de diccionarios. Asigna un ID único para la próxima receta que se agregue.
 - **guardar_recetas():** Guarda la lista de recetas en el archivo CSV.
 - **Operaciones CRUD:**
 - **get_all():** Devuelve todas las recetas en formato JSON.
 - **get_one(id):** Devuelve una receta específica identificada por su ID.
 - **create():** Crea una nueva receta a partir de datos JSON proporcionados, incrementa el ID para la próxima receta y guarda los cambios.
 - **update(id):** Actualiza una receta existente con nuevos datos y vuelve a entrenar el modelo de recetas.
 - **delete(id):** Elimina una receta identificada por su ID y actualiza el ID para las futuras recetas.



- **Modelo de Recomendación:**
 - **entrenar_modelo_recetas():** Entrena un modelo de vecinos más cercanos (KNN) utilizando los ingredientes de las recetas para sugerir recetas basadas en ingredientes proporcionados por el usuario.
- **Funciones Adicionales:**
 - **eliminar_tildes(texto):** Elimina las tildes de un texto dado, normalizando los caracteres.
 - **obtener_ingredientes(modelo_recetas):** Extrae y ordena los ingredientes únicos de todas las recetas.
 - **sugerir_recetas(modelo_recetas, ingredientes):** Sugiere recetas que contienen todos los ingredientes proporcionados, utilizando el modelo entrenado.

4.3 Archivo `index.html`

- **Descripción:** Define la estructura básica del buscador de recetas.
- **Funcionamiento:**
 - Incluye un campo de texto para que el usuario ingrese ingredientes y un contenedor para checkboxes con ingredientes disponibles.
 - Presenta botones para crear una nueva receta y para borrar selecciones previas.
 - Muestra los resultados de las recetas encontradas en función de los ingredientes ingresados.

4.4 Archivo `index.js`

- **Descripción:** Controla la interacción del usuario con el buscador de recetas.
- **Funcionamiento:**
 - Escucha el evento de entrada en el campo de ingredientes y sincroniza los checkboxes de ingredientes.



- Realiza una petición al backend para obtener la lista de ingredientes disponibles.
- Actualiza el textarea con los ingredientes seleccionados y obtiene las recetas basadas en los ingredientes introducidos.
- Maneja las acciones de modificar y eliminar recetas desde la interfaz.

4.5 Archivo crear.html

- **Descripción:** Proporciona la interfaz para crear nuevas recetas.
- **Funcionamiento:**
 - Contiene un formulario donde el usuario puede ingresar el nombre, ingredientes, elaboración y URL de la foto de la receta.
 - Incluye botones para volver al buscador de recetas y para agregar la receta.
 - Muestra la receta recién creada en un contenedor después de enviarla al backend.

4.6 Archivo crear.js

- **Descripción:** Gestiona la creación de nuevas recetas.
- **Funcionamiento:**
 - Escucha el evento de envío del formulario para agregar una nueva receta.
 - Captura los datos introducidos por el usuario (nombre, ingredientes, elaboración, URL de la foto).
 - Envía una petición POST al backend con la información de la receta.
 - Muestra la receta recién creada en la página después de recibir la respuesta del servidor.



4.7 Archivo `modificar.html`

- **Descripción:** Permite a los usuarios modificar recetas existentes.
- **Funcionamiento:**
 - Presenta un formulario que permite seleccionar qué campos de la receta se desean modificar (nombre, ingredientes, elaboración, URL de la foto).
 - Muestra campos de entrada según las opciones seleccionadas para facilitar la edición.
 - Incluye botones para volver al buscador de recetas y para enviar los cambios al backend.

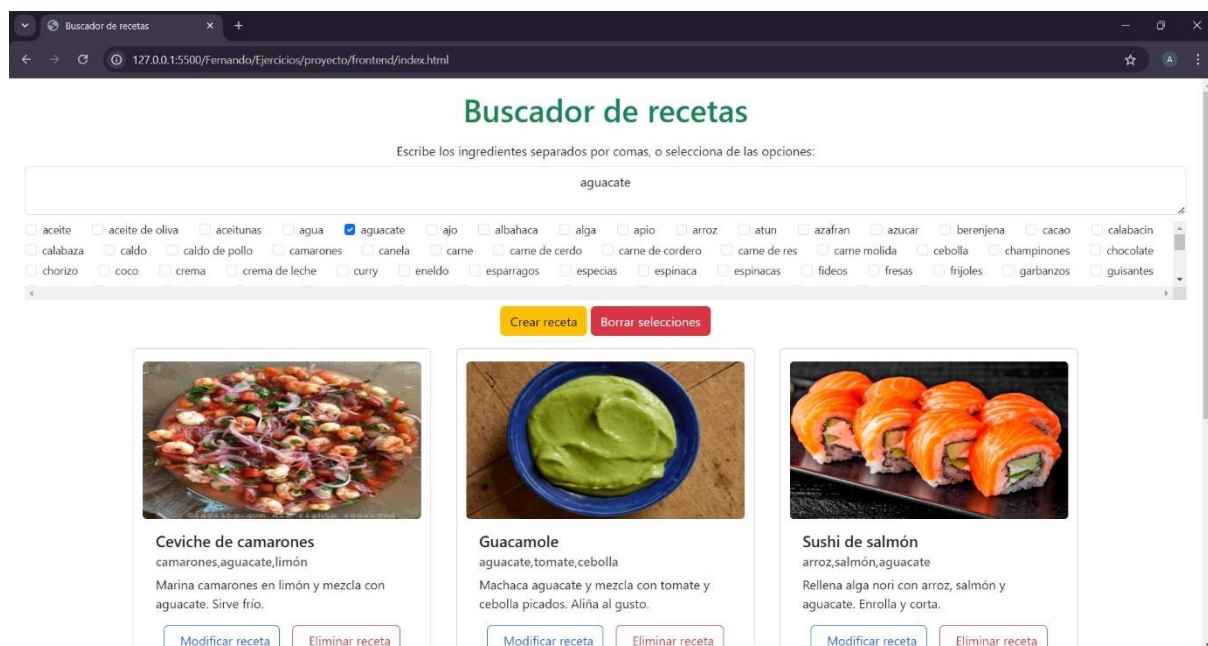
4.8 Archivo `modificar.js`

- **Descripción:** Controla la modificación de recetas existentes.
- **Funcionamiento:**
 - Escucha el evento de envío del formulario para modificar una receta.
 - Captura los campos modificados (nombre, ingredientes, elaboración, URL de la foto) según lo que el usuario haya seleccionado.
 - Envía una petición PUT al backend para actualizar la receta con los nuevos datos.
 - Maneja el almacenamiento del ID de la receta a modificar utilizando `localStorage`.

5. Funcionamiento del sistema

1. Gestión de recetas: El usuario puede crear, modificar, eliminar y listar recetas a través de la interfaz web.
2. Interacción con el servidor: Cada acción del usuario se traduce en solicitudes HTTP enviadas al servidor Flask, que maneja las operaciones sobre el archivo CSV donde se almacenan las recetas.
3. Búsqueda de recetas: El usuario puede introducir uno o varios ingredientes en un campo de búsqueda y enviar la solicitud al servidor.
4. Generación de recomendaciones: El servidor utiliza un modelo entrenado para sugerir recetas basadas en los ingredientes proporcionados y devuelve un listado de recetas coincidentes.
5. Visualización de resultados: Las recetas sugeridas se muestran en la interfaz web, permitiendo al usuario ver los detalles de cada receta, incluyendo ingredientes, elaboración y una imagen.

6. Capturas de pantalla





Cofinanciado por
la Unión Europea



GOBIERNO
DE ESPAÑA

MINISTERIO
DE INDUSTRIA
Y TURISMO



Fondos Europeos



Crear Receta

Nombre de la receta

Ejemplo: Pizza Margarita

Ingredientes (separados por comas)

Ejemplo: harina, tomate, queso...

Elaboración

Ejemplo: Hacer una base de masa fina, cubierta con salsa de tomate, mozzarella fresca y hojas de albahaca. Se hornea hasta que el queso se derrita y la masa esté crujiente.

URL de la foto

Ejemplo: https://example.com/foto.jpg

Buscar recetas Agregar Receta

Modificar Receta

Seleccionar opciones

- ☒ Modificar Nombre
- ☒ Modificar Ingredientes
- ☒ Modificar Elaboración
- ☒ Modificar URL de la foto

Nombre de la receta

Ejemplo: Pizza Margarita

Ingredientes (separados por comas)

Ejemplo: harina, tomate, queso...

Elaboración

Ejemplo: Hacer una base de masa fina, cubierta con salsa de tomate, mozzarella fresca y hojas de albahaca. Se hornea hasta que el queso se derrita y la masa esté crujiente.

URL de la foto

Ejemplo: https://example.com/foto.jpg

Buscar recetas Modificar Receta



Cofinanciado por
la Unión Europea



GOBIERNO
DE ESPAÑA
MINISTERIO
DE INDUSTRIA
Y TURISMO

EQI Escuela de
organización
Industrial



Fondos Europeos

SISTEMA NACIONAL DE
**GARANTÍA
JUVENIL**
SERVICIO PÚBLICO
DE EMPLEO ESTATAL | SEPE

Modificar Receta

Seleccionar opciones

- ☐ Modificar Nombre
- ☒ Modificar Ingredientes
- ☐ Modificar Elaboración
- ☐ Modificar URL de la foto

Ingredientes (separados por comas)

Ejemplo: harina, tomate, queso...

Buscar recetas Modificar Receta

Buscador de recetas

Escribe los ingredientes separados por comas, o selecciona de las opciones:

carne de cerdo

☐ aceite ☐ aceite de oliva ☐ aceitunas ☐ agua ☐ aguacate ☐ ajo ☐ albahaca ☐ alga ☐ apio ☐ arroz ☐ atun ☐ azafran ☐ azucar ☐ berenjena ☐ cacao ☐ calabacin ☐ calabaza ☐ caldo ☐ caldo de pollo ☐ camarones ☐ canela ☐ carne ☒ carne de cerdo ☐ carne de cordero ☐ carne de res ☐ carne molida ☐ cebolla ☐ champinones ☐ chocolate ☐ chorizo ☐ coco ☐ crema ☐ crema de leche ☐ curry ☐ eneldo ☐ espargagos ☐ especias ☐ espinaca ☐ espinacas ☐ fideos ☐ fresas ☐ frijoles ☐ garbanzos ☐ guisantes

Crear receta Borrar selecciones



Cerdo agridulce
carne de cerdo, piña, salsa barbacoa
Marina carne en salsa barbacoa, añade piña y hornea hasta que este cocido.

Modificar receta Eliminar receta

7. Consideraciones Adicionales

- **Persistencia de Datos:** La aplicación almacena recetas en un archivo CSV, asegurando que los datos persistan entre sesiones.
- **Manejo de Errores:** Se implementan respuestas adecuadas para errores comunes, como recetas no encontradas o falta de ingredientes en las solicitudes.
- **Interacción del Usuario:** Se ofrece una API RESTFUL, que permite la creación, lectura, actualización y eliminación de recetas, facilitando la interacción mediante solicitudes HTTP.
- **Limpieza de Datos:** Se incluye un proceso para eliminar tildes de los ingredientes, mejorando la consistencia en las búsquedas.

8. Bibliografía

- **Flask:** Microframework de Python utilizado para construir aplicaciones web. Facilita la creación de rutas y la gestión de solicitudes HTTP.
<https://flask.palletsprojects.com/>
- **Pandas:** Biblioteca de Python utilizada para el análisis de datos. Permite la manipulación y el análisis de datos en estructuras de datos tabulares como DataFrames.
<https://pandas.pydata.org/docs/index.html>
- **Scikit-learn:** Biblioteca de Python para aprendizaje automático. Se utiliza en esta aplicación para implementar el modelo de vecinos más cercanos (Nearest Neighbors) y la binarización de múltiples etiquetas (MultiLabelBinarizer).
<https://scikit-learn.org/stable/>
- **Flask-CORS:** Extensión de Flask que permite manejar CORS (Cross-Origin Resource Sharing) para permitir solicitudes de dominios externos.
<https://flask-cors.readthedocs.io/en/latest/>
- **Unicodedata:** Módulo de Python que proporciona acceso a la base de datos de caracteres Unicode. Se utiliza para eliminar acentos y normalizar cadenas de texto.
<https://docs.python.org/3/library/unicodedata.html>
- **CSV (Comma-Separated Values):** Formato de archivo utilizado para almacenar datos tabulares, facilitando la carga y el guardado de recetas en la aplicación.
https://en.wikipedia.org/wiki/Comma-separated_values
- **JavaScript Fetch:** Proporciona una forma sencilla y moderna de realizar solicitudes HTTP asíncronas.
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- **UUID de Python:** El módulo UUID de Python se utiliza para generar identificadores únicos, ideal para nombrar archivos de manera segura.
<https://docs.python.org/3/library/uuid.html>
- **HTML5 y JavaScript:** Son las tecnologías base para la construcción de interfaces web modernas.
<https://developer.mozilla.org/en-US/docs/Web/Guide/HTML/HTML5>
<https://developer.mozilla.org/en-US/docs/Web/JavaScript>

9. Conclusión

Este proyecto ilustra cómo se puede construir una aplicación web interactiva para la gestión de recetas utilizando Python y Flask, mostrando su potencial para resolver problemas cotidianos de manera eficiente y accesible. La implementación de un sistema que permite la creación, modificación y eliminación de recetas, junto con la capacidad de buscar recetas por ingredientes, brinda a los usuarios una herramienta práctica para organizar y acceder a sus recetas de cocina.

Python se destaca como una elección óptima para este proyecto debido a varias razones:

1. **Eficiencia en el manejo de datos:** Gracias a bibliotecas como pandas, el manejo y procesamiento de datos en formato CSV se realiza de manera sencilla y rápida, lo que permite cargar y guardar recetas de manera efectiva.
2. **Simplicidad en la construcción de APIs:** Flask, como microframework, facilita la creación de una API RESTFUL que responde a diferentes métodos HTTP, lo que permite que la aplicación maneje operaciones CRUD (Crear, Leer, Actualizar y Eliminar) de forma intuitiva y estructurada.
3. **Flexibilidad y escalabilidad:** La arquitectura modular de Flask permite que el sistema se amplíe fácilmente con nuevas características, como la integración de recomendaciones de recetas basadas en ingredientes, lo que enriquece la experiencia del usuario.
4. **Facilidad de integración de algoritmos de aprendizaje automático:** La incorporación de bibliotecas de machine learning, como scikit-learn, permite entrenar modelos que sugieren recetas según los ingredientes disponibles, ofreciendo un valor añadido a la aplicación y mejorando la interacción con el usuario.

5. **Amplia comunidad y recursos disponibles:** La gran comunidad de desarrolladores en torno a Python y Flask asegura que los desarrolladores tengan acceso a numerosos recursos y foros de soporte, lo que facilita la resolución de problemas y la implementación de nuevas funcionalidades.

En resumen, el uso de Python y Flask en este proyecto no solo permite la creación de una herramienta útil para la gestión de recetas, sino que también establece un fundamento sólido para futuras mejoras y expansiones. La capacidad de Python para manejar datos y su facilidad para construir aplicaciones web interactivas justifican plenamente su elección como tecnología principal en el desarrollo de esta aplicación.